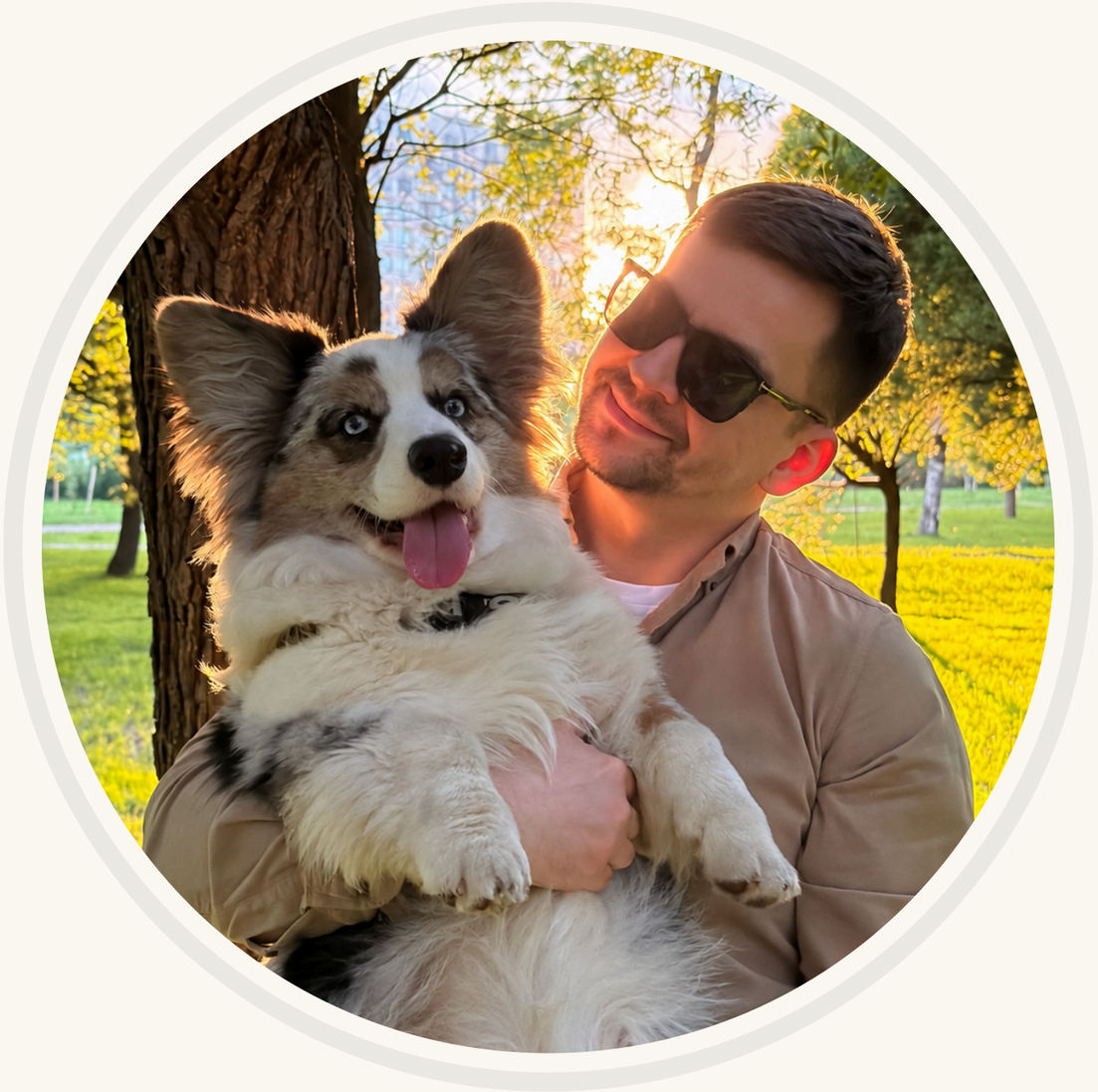
Ускоряем фронтенд с кодинг-агентом

Святослав Подмогаев

Святослав Подмогаев

Indie developer

15 лет во фронтенде



Можно ли поручить агенту поиск ускорения?

Дать код, способ измерения
и возможность пробовать

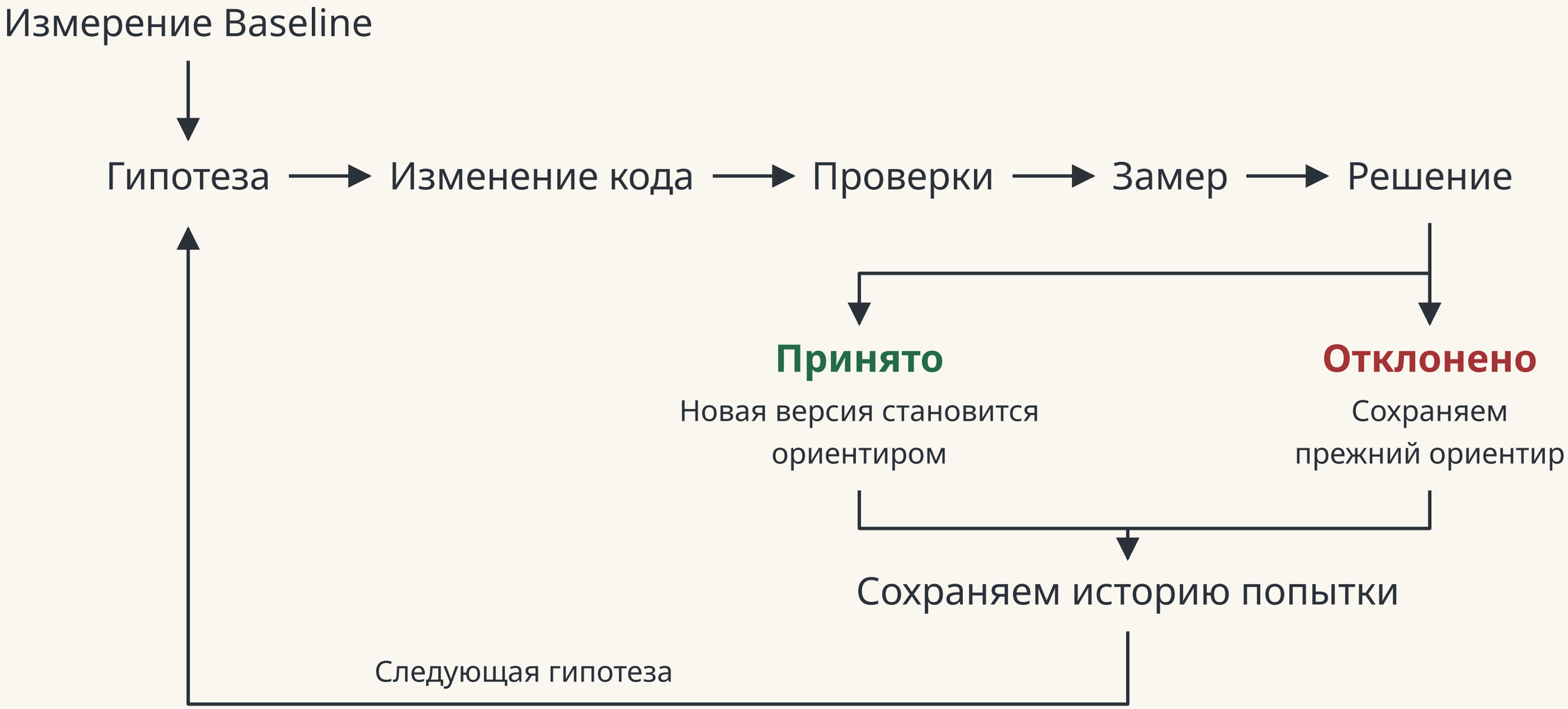
Оптимизация — поиск без рецепта

- Причина замедления не всегда очевидна
- Правдоподобная гипотеза может не дать ускорения
- Ускорение может нарушить поведение кода
- Каждая попытка требует правки, тестов и повторных замеров

Передаём перебор гипотез агенту

Разработчик задаёт	Цель, сценарии проверки и ограничения
Агент повторяет	Анализ кода, гипотеза, изменение
Пайплайн проверяет	Корректность и скорость; принимает или отклоняет результат

Автономный цикл



Проведём эксперимент: Day.js

Day.js — JavaScript-библиотека для работы с датами и временем

- **Зрелый проект:** широко используется,
  поддерживается мейнтейнерами и сообществом
- **Небольшая кодовая база:** удобно разбирать код и проверять изменения
- **Разные операции:** разбор и форматирование дат,
  арифметика и часовые пояса
- **Автоматическая проверка:** готовые тесты библиотеки
  и ожидаемые результаты сценариев

Сценарии эксперимента

Сценарий	Что выполняем
parse-iso	Разбор ISO-дат, проверка корректности и извлечение компонентов
parse-timestamps	Создание дат из timestamp, преобразование в ISO и Unix seconds
format-table	Форматирование даты и времени для ячеек таблицы
format-localized	Полная дата на английском, немецком и французском
parse-custom-strict	Строгий разбор DD/MM/YYYY HH:mm, включая некорректные даты
calendar-grid	Построение календарной сетки: 42 дня с границами недель и месяца
deadline-arithmetic	Сдвиг дат на месяцы, дни и недели; вычисление границ дня
filter-date-range	Отбор событий, попадающих в заданный диапазон дат
diff-dates	Разница между датами в днях, часах и дробных месяцах
relative-time	Относительное время от фиксированной даты на трёх языках
timezone-convert	Перевод UTC в часовые пояса Нью-Йорка, Лондона, Калькутты и Сиднея
timezone-parse	Разбор местной даты и времени в четырёх часовых поясах

Условия измерения

Нагрузка	12 сценариев с фиксированными данными
Повторы	30 замеров каждого сценария
Среда	Chromium через Playwright, новый контекст на каждый замер
Что измеряем	времени выполнения сценария
Порог принятия	На 1% относительно ориентира
Бюджет поиска	40 итераций и 4 часа

Как принимаем решение

Ориентир — последний принятый вариант, до первого улучшения — baseline

Итерация успешна: значение целевой функции снизилось относительно ориентира на **≥ 1%**, проверки пройдены, ограничения соблюдены

Иначе — **отклонить** и сохранить прежний ориентир

Как считаем целевую функцию

12 сценариев × 30 запусков = **360 замеров**

Медиана сценария

30 времён по возрастанию

$$m_i = \frac{t_{i,15} + t_{i,16}}{2}$$

Отношение к baseline

Для каждого сценария i

$$r_i = \frac{m_i \text{ новой версии}}{m_i \text{ baseline}}$$

Среднее отношений

Все сценарии равны по весу

$$C = \frac{r_1 + r_2 + \ldots + r_{12}}{12}$$

Принятая гипотеза: проверка даты

iteration-001

Убрать преобразование даты в строку из `isValid()`

Корректность даты можно проверить
по её числовому timestamp

Принятая гипотеза: изменение

iteration-001 · src/index.js

```
  isValid() {
-   return !(this.$d.toString() === C.INVALID_DATE_STRING)
+   return this.$d.getTime() < Infinity
  }
```

Замер и approve

iteration-001 · Ориентир: baseline

Было

1,0000

Стало

0,9286

−7,14%

Снижение целевой функции

Принято

Сценарий	Baseline, мс	001, мс	Изменение
parse-iso	2,385	1,660	-30,4%
parse-timestamps	1,968	2,072	+5,3%
format-table	4,380	3,722	-15,0%
format-localized	4,960	4,212	-15,1%
parse-custom-strict	7,307	5,990	-18,0%
calendar-grid	15,013	13,745	-8,4%
deadline-arithmetic	13,277	13,170	-0,8%
filter-date-range	3,442	3,465	+0,7%
diff-dates	15,012	15,053	+0,3%
relative-time	15,555	15,575	+0,1%
timezone-convert	21,342	20,830	-2,4%
timezone-parse	12,162	11,932	-1,9%

Отклонённая гипотеза: padStart

iteration-003

Быстрый путь для дополнения строки одним нулём

Меньше создания массивов и вызовов `join()` — меньше затрат на форматирование

Отклонённая гипотеза: изменение

iteration-003 · src/utils.js

```
  const padStart = (string, length, pad) => {
    const s = String(string)
    if (!s || s.length >= length) return string
+   if (pad === '0' && length === s.length + 1) return `0${string}`
    return `${Array((length + 1) - s.length).join(pad)}${string}`
  }
```

Замер и reject

iteration-003 · Ориентир: iteration-002

Было

0,8916

Стало

0,8859

−0,64%

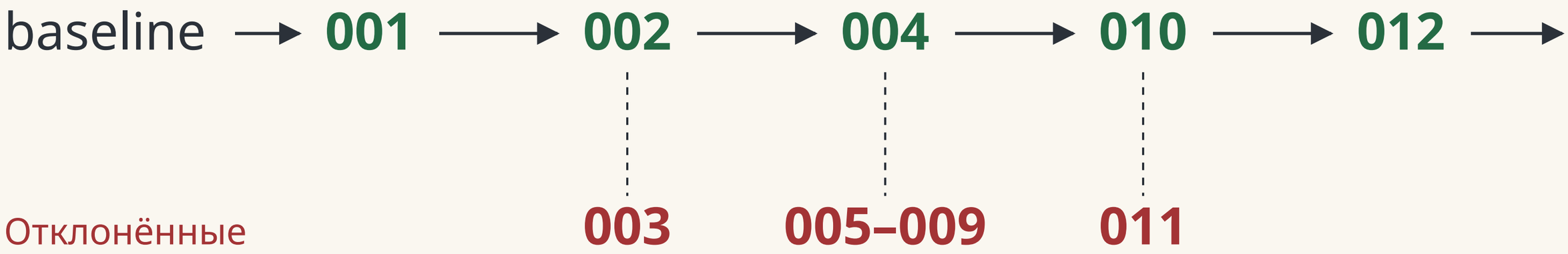
Снижение целевой функции

Отклонено

Порог: 1%

Сценарий	002, мс	003, мс	Изменение
parse-iso	1,625	1,655	+1,8%
parse-timestamps	2,038	2,018	-1,0%
format-table	3,267	3,200	-2,1%
format-localized	3,735	3,743	+0,2%
parse-custom-strict	5,452	5,305	-2,7%
calendar-grid	13,147	12,872	-2,1%
deadline-arithmetic	13,465	13,217	-1,8%
filter-date-range	3,503	3,477	-0,7%
diff-dates	14,662	14,792	+0,9%
relative-time	15,072	15,152	+0,5%
timezone-convert	19,880	20,015	+0,7%
timezone-parse	11,583	11,427	-1,3%

Как накапливаются изменения



> Каждая попытка начинается с последней принятой версии кода

Результаты: весь прогон

−38,3%

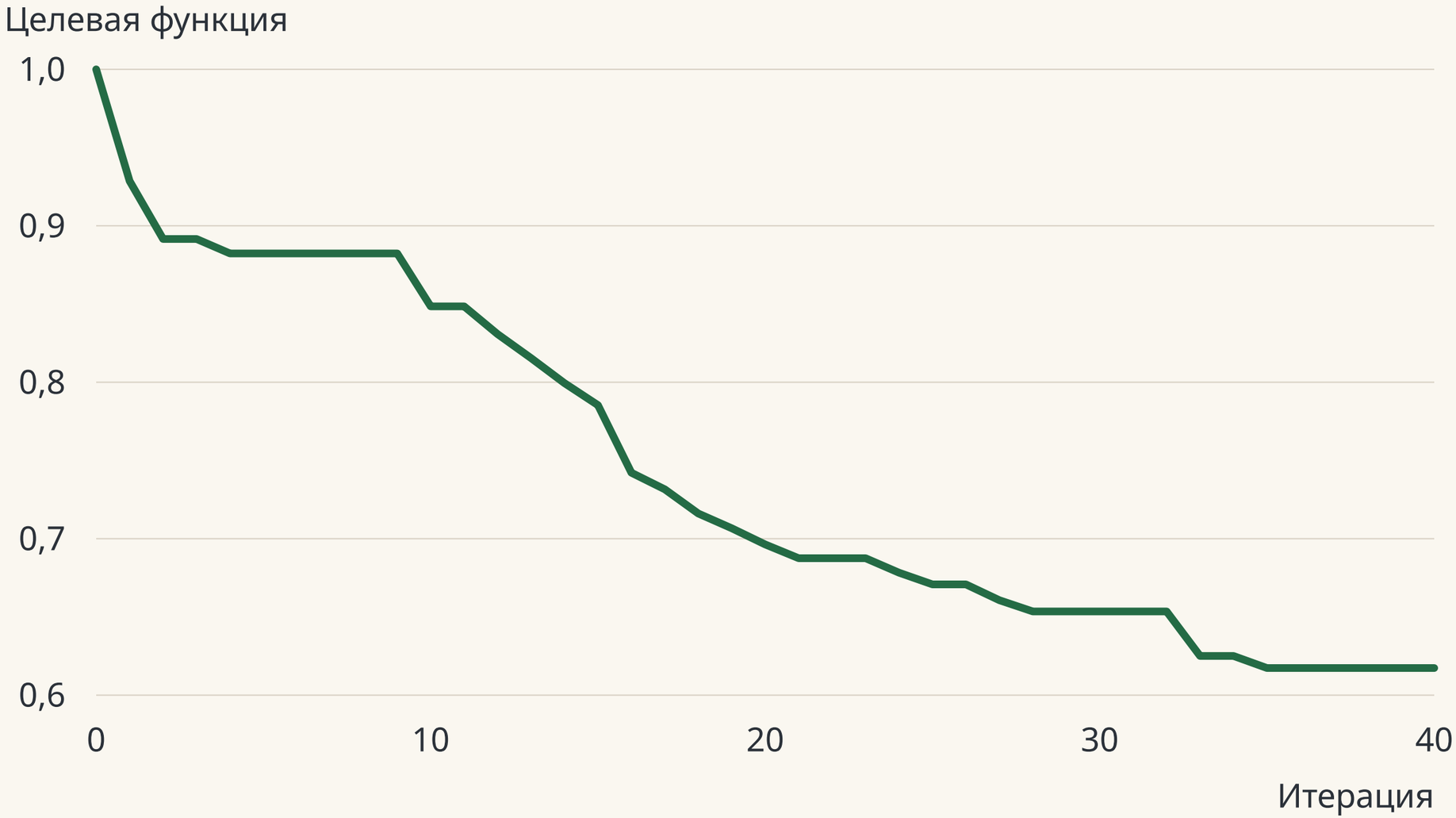
Снижение целевой функции

2 ч 5 мин

Время выполнения

+161 −66

Строк кода
Baseline → 035



Результаты: по сценариям

Сценарий	Baseline, мс	iteration-035, мс	Изменение
parse-iso	2,385	1,565	-34,4%
parse-timestamps	1,968	1,930	-1,9%
format-table	4,380	2,482	-43,3%
format-localized	4,960	3,352	-32,4%
parse-custom-strict	7,307	4,305	-41,1%
calendar-grid	15,013	4,233	-71,8%
deadline-arithmetic	13,277	8,540	-35,7%
filter-date-range	3,442	2,743	-20,3%
diff-dates	15,012	6,948	-53,7%
relative-time	15,555	7,665	-50,7%
timezone-convert	21,342	9,582	-55,1%
timezone-parse	12,162	9,885	-18,7%

Ограничения эксперимента

- Ограниченный набор сценариев

Ограничения эксперимента

- Ограниченный набор сценариев
- Равные веса сценариев

Ограничения эксперимента

- Ограниченный набор сценариев
- Равные веса сценариев
- Единственная метрика

Ограничения эксперимента

- Ограниченный набор сценариев
- Равные веса сценариев
- Единственная метрика
- Неполная гарантия совместимости

Ограничения эксперимента

- Ограниченный набор сценариев
- Равные веса сценариев
- Единственная метрика
- Неполная гарантия совместимости
- Без оценки статистической значимости

Не только фронтенд

Подход применим там, где результат
можно автоматически оценить

- autoresearch Андрея Карпатого:

  агент меняет обучение языковой модели, запускает эксперимент
  и сохраняет изменения, которые улучшили качество

- AlphaEvolve — Google DeepMind:

  агент предлагает алгоритмы, проверяет их автоматически и развивает лучшие варианты

- Eureka — NVIDIA и университеты:

  агент подбирает, за что поощрять робота при обучении, чтобы тот лучше выполнял задачу

- Поиск квантовых схем с LLM:

  агент ищет схемы, которые правильно выполняют вычисление и требуют меньше операций и кубитов

Главный вывод

Мы можем автоматизировать не только
выполнение известного решения,
но и поиск решения, которого ещё не знаем

Посмотреть поближе



github.com/prostohz/piterjs-perf-talk